ZK from Zero on Cardano

2025

by ElRaulito

I FOUNDATIONS OF ZERO-KNOWLEDGE 2

1 INTRODUCTION TO ZERO-KNOWLEDGE PROOFS 3

1.1 What are Zero-Knowledge Proofs? 3

1.2 Core Concepts 4

1.3 Types of Zero-Knowledge Proof Systems 5

1.4 ZK on Cardano 5

II ORIGINS AND EVOLUTION 6

2 THE HISTORY OF ZK 7

2.1 The Origins of Zero-Knowledge 7

2.2 From Theory to Practice 7

2.3 The SNARK Revolution 8

2.4 Modern ZK: STARKs and Beyond 8

2.5 ZK Arrives on Cardano 9

III ZK IN WEB3: PRIVACY COINS AND BEYOND 11

3 ZK IN THE WILD 12

3.1 Why Web3 Needs ZK 12

3.2 Zcash 12

3.3 Monero 14

3.4 Other ZK Applications in Web3 15

3.5 Midnight 16

IV ZK FOR COMPRESSION AND PRIVACY 21

4 PERFORMANCE, PRIVACY AND TRADE-OFFS 22

4.1 ZK for Data Minimisation and Proof Compression 22

4.2 Succinctness and Scalability 23

4.3 Privacy by Design: Applications and Limitations 24

4.4 The Trade-offs: Performance vs. Privacy 25

V BUILDING ZK ON CARDANO: CIRCUITS AND VERIFICATION 27

5 CIRCOM, SNARKJS, AND AIKEN 28

5.1 The Circuit Pipeline 28

5.2 Writing a Circuit in Circom 28

5.3 The Trusted Setup Ceremony 30

5.4 Generating and Verifying a Proof 33

5.5 Preparing the Verification Key for Cardano 34

5.6 On-Chain Verification in Aiken 35

5.7 Janus Wallet: Putting It All Together 36

VI YOUR FIRST ZK CONTRACT ON CARDANO 39

6 PASSWORD-LOCKED UTXO WITH GROTH16 40

6.1 What We Are Building 40

6.2 Project Setup 40

6.3 The Circuit 41

6.4 Generating `input.json` 42

6.5 The Trusted Setup 44

6.6 Generating the Proof 45

6.7 Compressing Points for Cardano 45

6.8 The Groth16 Verifier Library 47

6.9 The Aiken Validator 49

6.10 Testing with Aiken 50

6.11 Off-Chain Code with MeshJS 52

6.12 End-to-End Summary 54

6.13 How This Differs from Janus Wallet 54

FOUNDATIONS OF ZERO-KNOWLEDGE

1. INTRODUCTION TO ZERO-KNOWLEDGE PROOFS

1.1. WHAT ARE ZERO-KNOWLEDGE PROOFS?

All of us have heard about Zero-Knowledge, but what is it exactly? In my experience, many projects have abused this term — using it only to hide something already known, like multi-signature protocols, or centralising data and value behind a web2 backend while claiming ZK legitimacy.

A telling example: a project once claimed to make all transactions anonymous. You would send stablecoins to them, tell them the intended receiver, and they would forward the funds bundled with other transfers so the origin was obscured. There is no ZK in that process — it is simply a centralised mixer.

Where's Waldo and the Hash Lock Analogy

The example that first made ZK click for me is **Where's Waldo** (known as *Where's Wally* outside North America). Imagine you have found Waldo on a busy page but you want to prove to a friend that you know where he is without pointing at him. You could take a very large sheet of paper, cut a small hole over Waldo, and show just his face through the hole. Your friend sees Waldo and is convinced you know the location — without learning anything else about the page.

Using secrets in blockchains was already possible in Bitcoin with **hash-lock scripts**: you choose a password known only to you, hash it, and lock funds to that hash. Anyone who can reveal the pre-image unlocks the coins.

> Hash Time-Locked Contracts (HTLCs) are the technology behind cross-chain atomic swaps — used by protocols such as *Fluid Tokens* — and underpins payment routing on the Bitcoin Lightning Network.

The critical limitation of hash locks is that revealing the password burns it: once the pre-image is public, that address is permanently unsafe because anyone now knows the secret tied to it. With ZK you can **prove that you know the password without ever revealing it**. That is the real magic.

ZK as Compression, Not Just Privacy

Most people stop at the privacy angle. ZK is far more useful as a **compression tool**, and this is arguably its most important application in blockchain today.

ZK ROLLUPS (GENERAL CASE).

An L2 network may process thousands of transactions per second. Posting every transaction to L1 would be prohibitively expensive. Instead, the rollup posts only a ZK proof attesting that all those transactions were correctly signed by the users involved.

An L1 smart contract verifies the proof — a single compact check replaces thousands of individual verifications.

> **Thousands of L2 transactions ⟶ one ZK proof on L1.**
> The on-chain contract only verifies the proof. Nobody can lie about what happened.

THE CARDANO USE CASE.

When you interact with a Cardano DEX today, the contract executes all calculations on-chain: it checks amounts, validates prices, and enforces invariants for *every* order, consuming script budget. With ZK the flow changes completely:

1. You build the transaction and perform all calculations *off-chain* — the amounts to receive, slippage bounds, fee splits.
2. You attach to your transaction a ZK proof of those computations.
3. The DEX contract checks only that the proof is valid. It does not re-execute the math.

Because verification is orders of magnitude cheaper than re-computation, a single transaction can batch hundreds of orders. You see? ZK is being used to **compress computation**, not to hide anything.

1.2. CORE CONCEPTS

Before going further it is worth fixing the three properties that every honest ZK system must satisfy.

- **Completeness.** If the statement is true and the prover is honest, the verifier will always be convinced.
- **Soundness.** If the statement is false, no cheating prover can convince the verifier (except with negligible probability).
- **Zero-Knowledge.** The verifier learns nothing beyond the truth of the statement. The proof reveals no information about the secret witness.

In every ZK protocol there are two parties:

- **Prover** — knows a secret (the *witness*) and wants to convince the verifier without revealing it.
- **Verifier** — checks the proof and decides whether to accept it.

Interactive vs. Non-Interactive Proofs

Early ZK proofs were *interactive*: the verifier sends random challenges and the prover responds, repeating rounds until the verifier is statistically convinced. This is elegant in theory but impractical on a blockchain where there is no live back-and-forth.

The **Fiat–Shamir heuristic** (1986) transforms interactive protocols into *non-interactive* ones: the prover generates the verifier's challenges themselves by hashing the transcript

so far. The resulting proof can be verified by anyone, at any time, without interaction — exactly what on-chain verification requires.

1.3. TYPES OF ZERO-KNOWLEDGE PROOF SYSTEMS

The ZK landscape has exploded in the past decade. Here is a practical map of the main families.

- **zk-SNARKs** (Succinct Non-interactive Arguments of Knowledge) — very small proofs and fast verification, but require a trusted setup ceremony. Groth16 and PLONK are the dominant constructions. Used in Zcash, Ethereum's zkSync and Scroll.
- **zk-STARKs** (Scalable Transparent Arguments of Knowledge) — no trusted setup, post-quantum secure, but larger proof sizes. Used by StarkWare and StarkNet.
- **Bulletproofs** — no trusted setup, compact range proofs, but verification is slower and scales linearly with circuit size. Used in Monero and Mimblewimble.
- **PLONK and universal SNARKs** — use a single trusted setup that works for *any* circuit up to a given size, removing the need for per-circuit ceremonies.

System	Trusted Setup	Proof Size	Verification
Groth16	Per-circuit	∼200 bytes	Very fast
PLONK	Universal	∼500 bytes	Fast
STARKs	None	∼45–200 KB	Moderate
Bulletproofs	None	∼1 KB	Slow (linear)

1.4. ZK ON CARDANO

Cardano's **eUTxO model** is well-suited to ZK verification: a script validator receives the proof as part of the redeemer or datum and checks it in a single deterministic execution. Because Cardano contracts run off-chain before submission, the prover can generate the proof locally and attach it — the on-chain script only verifies.

With **Plutus V3**, Cardano gained native cryptographic primitives for the BLS12-381 elliptic curve pairing:

- `bls12_381_G1_add`, `bls12_381_G2_add` — elliptic curve point addition in $\mathbb{G}_1$ and $\mathbb{G}_2$.
- `bls12_381_millerLoop`, `bls12_381_finalVerify` — pairing operations needed to verify Groth16 proofs on-chain.

These primitives are also exposed in **Aiken**'s standard library under `aiken/crypto/bls12_381`, making it practical to write on-chain Groth16 verifiers in a high-level language.

The goal of this book is to start from the very basics — proving to someone that you know a value and can therefore spend a UTxO — and build up to more complex applications: compressing off-chain computation, and eventually constructions similar to privacy protocols (studied here for academic purposes only).

ORIGINS AND EVOLUTION

2. THE HISTORY OF ZK

2.1. THE ORIGINS OF ZERO-KNOWLEDGE

Zero-Knowledge proofs were invented in **1985** by Shafi Goldwasser, Silvio Micali, and Charles Rackoff at MIT. Their paper — *The Knowledge Complexity of Interactive Proof Systems* — introduced the concept of an interactive proof where a prover convinces a verifier of the truth of a statement while revealing nothing beyond that truth. The paper won the Gödel Prize in 1993 and is widely considered one of the most influential works in the history of cryptography.

At the time, the result was purely theoretical. The proof systems described required many rounds of interaction, were expensive to compute, and existed only to answer a complexity-theoretic question: *how much "knowledge" does a proof leak?* The answer, in some cases, was: none.

2.2. FROM THEORY TO PRACTICE

The Schnorr Protocol

A landmark step toward practicality came with Claus-Peter Schnorr's identification scheme (1989). The **Schnorr protocol** is a three-move sigma protocol:

1. The prover commits to a random value.
2. The verifier sends a random challenge.
3. The prover responds with a value that combines the challenge with their secret.

The verifier can check the response without learning the secret. Sigma protocols like this are the ancestors of almost every practical ZK system in use today.

The Fiat-Shamir Heuristic

In 1986, Amos Fiat and Adi Shamir showed how to convert any sigma protocol into a **non-interactive** proof by replacing the verifier's random challenge with a hash of the prover's commitment. The resulting proof can be generated offline and verified by anyone — no live interaction needed. This single insight is what makes ZK proofs viable on blockchains.

Schnorr Signatures on Cardano with FROST

Schnorr signatures eventually became a production cryptographic primitive. Today you can use them on Cardano through the **FROST** threshold signature protocol, accessible at `https://www.frost.express/`. FROST allows a group of participants to collaboratively sign a message with a Schnorr signature in a way that is indistinguishable from a single-signer scheme on-chain, with no single party holding the full private key.

2.3. THE SNARK REVOLUTION

Through the 2000s and early 2010s, researchers worked to make ZK proofs efficient enough for real applications. The breakthrough came in 2013–2016 with a family of constructions called **zk-SNARKs**.

TRUSTED SETUP AND GROTH16.

The most efficient SNARK construction is **Groth16** (Jens Groth, 2016). It produces proofs of only ∼200 bytes that verify in milliseconds — fast enough for on-chain use. The catch is a *trusted setup ceremony*: a group of participants must generate public parameters, and if any participant keeps their secret "toxic waste", they could forge proofs. In practice, multi-party ceremonies with hundreds of participants make this risk negligible.

ZCASH AND REAL-WORLD DEPLOYMENT.

Zcash (2016) was the first large-scale deployment of zk-SNARKs. It uses Groth16 to let users make shielded transactions: the blockchain verifies a proof that the sender has enough funds and the transaction is balanced, without revealing amounts or addresses.

UNIVERSAL SETUPS.

Circuit-specific trusted setups are impractical for general use. **PLONK** (2019) and **Marlin** introduced *universal* setups: a single ceremony valid for any circuit up to a given size. This enabled the modern ecosystem of general-purpose zkVMs and zkEVMs.

2.4. MODERN ZK: STARKS AND BEYOND

STARKS.

Eli Ben-Sasson and colleagues at the Technion introduced **zk-STARKs** in 2018. STARKs require *no trusted setup* — all parameters are public and derived from collision-resistant hash functions. They are also believed to be post-quantum secure. The trade-off is proof size: a STARK proof can be tens to hundreds of kilobytes versus a few hundred bytes for a SNARK.

StarkWare built an entire ecosystem around STARKs: *StarkEx* powers major DeFi protocols, and *StarkNet* is a permissionless L2 on Ethereum.

ZK ROLLUPS.

The rollup model took off around 2020–2022. **zkSync**, **Scroll**, and **Polygon zkEVM** all use ZK proofs to compress thousands of Ethereum transactions into a single on-chain verification, achieving Ethereum security with a fraction of the gas cost.

RECENT DEVELOPMENTS.

The frontier is moving fast: *folding schemes* like **Nova** (2021) allow recursive proof composition with near-zero overhead; *lookup arguments* like Plookup dramatically reduce the cost of proving table lookups and bit operations; and *zkVMs* such as RISC Zero let developers write programs in Rust and get a ZK proof of correct execution essentially for free.

2.5. ZK ARRIVES ON CARDANO

Plutus V3 and BLS12-381 Primitives

With the Chang hard fork (2024), Cardano gained **Plutus V3** and a set of native BLS12-381 elliptic curve primitives. These allow on-chain verification of Groth16 proofs without any workarounds. The primitives are available in Aiken's standard library:

- `aiken/crypto/bls12_381/g1` — operations on the $\mathbb{G}_1$ group
- `aiken/crypto/bls12_381/g2` — operations on the $\mathbb{G}_2$ group
- Pairing operations: `miller_loop` and `final_verify`

See the full API at `https://aiken-lang.github.io/stdlib/aiken/crypto/bls12_381/g1.html`.

Janus Wallet

The most complete open-source ZK project on Cardano today is **Janus Wallet**, available at:

`https://github.com/leobel/janus-wallet`

Janus demonstrates a working Groth16 verifier on Cardano: a user generates a ZK proof off-chain that they know a secret, and the on-chain Aiken validator verifies the proof and releases the UTxO. It is the closest thing Cardano has to a production-grade ZK smart contract today.

What This Book Covers

The rest of this book is structured as a progressive journey:

1. **Starting from basics** — proving knowledge of a secret value and using a ZK proof to spend a UTxO on Cardano.
2. **Compression** — using off-chain computation with on-chain verification to batch many operations into a single transaction.
3. **Advanced constructions** — privacy-preserving patterns studied for academic purposes. All material here is for educational use. This book does not promote protocols designed to obscure transaction data.

ZK IN WEB3: PRIVACY COINS AND BEYOND

3. ZK IN THE WILD

3.1. WHY WEB3 NEEDS ZK

Blockchain is beautiful precisely because it is transparent — but that same transparency can be turned against you.

In 2022, Tornado Cash was placed on the US government's sanctions list. If your address had interacted with it in any transaction, exchanges could freeze your assets. Worse, someone could *send* you a transaction through Tornado Cash and get your address blacklisted without your consent. Once your address is linked to your real identity, every trade, transfer, and incoming payment becomes traceable. Dirty money from hacks could be sent to you on purpose.

This is why on-chain privacy matters — not to hide wrongdoing, but to protect ordinary users from targeted attacks, surveillance, and financial coercion.

Bitcoin Privacy Practices

Bitcoin has no built-in privacy layer, but the community has developed a set of good practices over the years. I believe these can be replicated straightforwardly on Cardano and are worth stating clearly:

- **Never reuse an address.** Bitcoin and Cardano both support deriving unlimited addresses from a single seed. Each payment should receive a fresh address.
- **Avoid round-number amounts and self-change.** Sending exactly 1 BTC and returning change to yourself are fingerprints that chain analytics tools look for.
- **Use multiple inputs and outputs.** A transaction that mixes several UTxOs from different addresses and fans out to several recipients is harder to trace than a one-to-one transfer.
- **Isolate KYC funds.** If you received ADA from a KYC exchange, keep that UTxO entirely separate from addresses you want to keep private. Mixing them collapses your privacy instantly.

These practices reduce linkability but cannot eliminate it. For stronger guarantees you need a cryptographic construction — which is where ZK enters the picture.

3.2. ZCASH

Zcash gained mainstream attention during the Silk Road era: because Silk Road was a black market where participants needed payment privacy, a coin with strong cryptographic guarantees was attractive. Unfortunately the listing of Zcash was front-run by the US government's take-down and arrest of Ross Ulbricht, the founder of Silk Road, before Zcash could see widespread use there. Ross spent ten years in prison; the last US president promised to pardon him during the campaign and ultimately did — he is now a free man.

> The story of Ross Ulbricht is a reminder that financial privacy and personal freedom are deeply connected. Regardless of your view of Silk Road, the case set important legal precedents about online speech, sentencing, and the use of blockchain evidence.

How Zcash Works

Zcash uses **zk-SNARKs** (specifically the Groth16 construction) to enable shielded transactions. Every transaction on Zcash involves two possible address types:

- **Transparent addresses** (t-addresses) — behave like Bitcoin addresses; all amounts and addresses are public.
- **Shielded addresses** (z-addresses) — balances and transaction details are hidden behind a ZK proof.

When you send ZEC from one shielded address to another, the transaction contains a proof that:

1. You own a note (a UTXO) with sufficient value.
2. The total value in equals the total value out (no inflation).
3. You are not double-spending.

The verifier (the entire network) checks only the proof. Amounts, sender, and receiver remain invisible. The construction relies on a **trusted setup ceremony** — a multi-party computation that generates the public parameters. If every participant in the ceremony destroys their toxic waste, the parameters are safe. Zcash ran the Sapling ceremony in 2018 with hundreds of participants.

> **Zcash in one sentence:** a proof that your transaction is valid, without revealing who you are, how much you sent, or who received it.

The main criticism of Zcash historically has been low adoption of shielded addresses. Most ZEC on exchanges sits in transparent addresses, which undermines the anonymity set. Shielded usage has grown since the Sapling upgrade, but full privacy still requires both sender and receiver to use z-addresses.

3.3. MONERO

Monero takes a different philosophical stance from Zcash: **privacy by default for every transaction**, not opt-in. There are no transparent addresses. Every Monero transaction is private.

Monero achieves this through three complementary mechanisms:

- **Ring Signatures.** When you spend XMR, your transaction is signed by a ring of n possible signers drawn from the UTXO set. An observer cannot tell which ring member is the true sender.
- **Stealth Addresses.** Each transaction generates a one-time destination address derived from the recipient's public key. Only the recipient can link it to their wallet.
- **Confidential Transactions (RingCT / Bulletproofs).** Transaction amounts are hidden using Pedersen commitments. A Bulletproof proves that all amounts are positive and that inputs equal outputs, without revealing the values. Bulletproofs require no trusted setup.

Feature	Zcash (ZEC)	Monero (XMR)
Launch Year	2016	2014
Founders	Zooko Wilcox-O'Hearn & Electric Coin Company	Nicolas van Saberhagen (pseudonymous)
Privacy Mechanism	zk-SNARKs (zero-knowledge proofs)	Ring Signatures, Stealth Addresses, RingCT
Privacy Type	Optional (transparent or shielded)	Mandatory (always private)
Consensus Mechanism	Proof-of-Work (Equihash algorithm)	Proof-of-Work (RandomX algorithm)
Total Supply	21 million ZEC	~18.4 million XMR with tail emission
Development Organizations	Electric Coin Company & Zcash Foundation	Monero Core Team & Community Developers
Smart Contract Support	Limited (possible via cross-chain integrations)	None (focused purely on privacy transactions)
Regulatory Accessibility	Higher due to optional transparency	Lower due to enforced anonymity
2025 Market Capitalization	$10.8 billion USD	$7.8 billion USD

Datawallet

Zcash vs. Monero at a glance:

	Zcash	Monero
Privacy model	Opt-in (shielded)	Mandatory (always on)
Proof system	zk-SNARK (Groth16)	Bulletproofs (no ZK circuit)
Trusted setup	Yes (Sapling)	No
Anonymity set	All shielded UTxOs	Ring of n UTxOs
Proof size	~200 bytes	~2 KB

Monero's mandatory privacy means every user contributes to the anonymity set, which is a fundamental advantage. The trade-off is that ring signatures scale linearly, and proving range correctness with Bulletproofs is slower than a SNARK verifier.

3.4. OTHER ZK APPLICATIONS IN WEB3

ZK proofs have spread far beyond privacy coins. Here are the most important application domains and projects.

Mina Protocol

Mina describes itself as the world's lightest blockchain. Instead of storing the full chain history, every node stores only a **constant-size ZK proof** (currently 22 KB) that the entire chain is valid. This proof is recursive: when a new block is produced, the block producer generates a proof that the new state is valid *and* that the previous proof was valid. The chain compresses down to a single proof of everything that ever happened.

This is a stunning result: syncing a Mina node takes seconds regardless of chain age, because you only verify one proof.

Aztec

Aztec is a privacy-first ZK rollup on Ethereum. Its **Noir** language lets developers write circuits in a high-level, Rust-like syntax. Aztec's architecture separates:

- **Private functions** — executed entirely client-side, producing ZK proofs, with no on-chain trace.
- **Public functions** — executed by the sequencer, visible on-chain.

Applications on Aztec can mix both: a private transfer function hides the sender and amount, while a public function updates a visible registry. Noir compiles to ACIR (Abstract Circuit Intermediate Representation), which multiple backends can prove.

Polygon zkEVM

Polygon zkEVM is a ZK rollup that is **EVM-equivalent**: Ethereum smart contracts and tooling work without modification. Every batch of transactions is accompanied by a zk-SNARK proof that the EVM state transition was correct. The L1 Ethereum contract verifies the proof in a single transaction, achieving Ethereum-level security with dramatically lower gas costs per operation.

StarkNet

StarkNet uses **zk-STARKs** (no trusted setup, post-quantum secure) and its own language, Cairo, which compiles to a provable virtual machine. STARKs produce larger proofs than SNARKs but scale better for very large computations and avoid the trusted-setup risk entirely.

3.5. MIDNIGHT

Midnight is a **data-protection blockchain** developed by Input Output Global (IOG) as a partner chain to Cardano. Its goal is to let developers build applications that can *selectively disclose* information while keeping everything else private, using zero-knowledge proofs as the underlying engine.

What Midnight Enables

- **Private Trading.** Decentralized exchanges where order size and execution timing remain hidden from the public ledger.
- **Confidential DeFi.** Private lending, portfolio management, and liquidity pools where balances are shielded.
- **Compliant Identity (ZK-KYC).** Prove that you are over 18, or that you are an accredited investor, without revealing your passport or income documents to every counterparty.
- **Shielded and Unshielded Tokens.** Developers can create tokens that keep balances private by default, or public when the application requires it.
- **Compact Smart Contracts.** Midnight uses a TypeScript-based language for writing privacy-preserving applications, keeping the developer experience accessible.

How Midnight Infrastructure Works

Running on Midnight requires two services working together:

- **Node** — connects to the Midnight network, participates in consensus, and submits transactions.
- **Indexer** — reads data from the node and exposes it through a **GraphQL API**.

> The biggest architectural difference from Cardano's REST-based APIs is that the Midnight indexer speaks **GraphQL**. Instead of hitting specific endpoints, you write a query describing exactly what you want, and the API returns only that data. This means fewer round trips and more flexibility, but requires a different mental model if you are used to REST.

Blockfrost hosts both the node and indexer as a managed service across three environments:

Environment	Use
Mainnet	Production
Preview	Testing
Preprod	Testing

The three service endpoints for Mainnet are:

Service	Endpoint
Indexer API	`https://midnight-mainnet.blockfrost.io/api/v0/`
WebSocket	`wss://midnight-mainnet.blockfrost.io/api/v0/ws`
Node RPC	`https://rpc.midnight-mainnet.blockfrost.io`

The Indexer API handles queries for blocks, transactions, contracts, and wallet events. The WebSocket supports real-time subscriptions for block streaming, contract actions, and ledger events. The Node RPC provides direct JSON-RPC access for wallet providers and transaction submission.

Authentication

There are two ways to authenticate with Blockfrost's Midnight endpoints:

- **Header:** pass your API key as `project_id` in the request header (same as Cardano).
- **Query parameter:** append `?project_id=YOUR_PROJECT_ID` to the URL — useful for clients that cannot set custom headers, including Midnight.js itself.

When configuring Midnight.js to use Blockfrost, pass the full URL with the project ID embedded:

```
https://midnight-mainnet.blockfrost.io/api/v0/?project_id=YOUR_PROJECT_ID
```

Getting Started with Midnight.js

Midnight.js is the official SDK for building on Midnight. It handles contract deployment, private state management, ZK proof generation, and transaction submission.

PREREQUISITES.

- Node.js 22.x or higher
- Docker (required to run the local proof server)
- A Blockfrost account with a Midnight project (free tier available)

INSTALLATION.

Install the full SDK barrel package:

```
npm install @midnight-ntwrk/midnight-js
```

Or install individual packages if you only need specific capabilities:

```
npm install @midnight-ntwrk/midnight-js-types
npm install @midnight-ntwrk/midnight-js-contracts
npm install @midnight-ntwrk/midnight-js-network-id
```

STEP 1 – SET THE NETWORK.

Before any operation, configure which network you are targeting:

```
import { setNetworkId } from '@midnight-ntwrk/midnight-js-network-id';

setNetworkId('preprod'); // or 'mainnet', 'preview'
```

STEP 2 – CONFIGURE PROVIDERS.

Midnight.js uses a modular provider model. Each provider handles one concern:

```
import { levelPrivateStateProvider }
  from '@midnight-ntwrk/midnight-js-level-private-state-provider';
import { indexerPublicDataProvider }
  from '@midnight-ntwrk/midnight-js-indexer-public-data-provider';
import { FetchZkConfigProvider }
  from '@midnight-ntwrk/midnight-js-fetch-zk-config-provider';
import { httpClientProofProvider }
  from '@midnight-ntwrk/midnight-js-http-client-proof-provider';

const blockfrostUrl =
  'https://midnight-preprod.blockfrost.io/api/v0'
  + '?project_id=YOUR_PROJECT_ID';

const providers = {
  // Encrypted local storage for your private state
  privateStateProvider: levelPrivateStateProvider({
    storePath: './my-private-state'
  }),
  // GraphQL queries to the Midnight indexer
  publicDataProvider: indexerPublicDataProvider(blockfrostUrl),
  // Fetches ZK artifacts (prover keys, verifier keys)
  zkConfigProvider: new FetchZkConfigProvider('/zk-artifacts'),
  // Sends circuits to the proof server for proving
  proofProvider: httpClientProofProvider('http://localhost:6300')
};
```

The proof server runs locally in Docker. It receives your circuit inputs and returns a ZK proof that can be attached to a transaction. Your private inputs never leave your machine.

STEP 3 – DEPLOY A CONTRACT.

```
import { deployContract }
  from '@midnight-ntwrk/midnight-js-contracts';
import { compiledContract } from './your-contract';

const deployed = await deployContract(providers, {
  compiledContract,
  privateStateId: 'my-state',
  initialPrivateState: { counter: 0n }
});

console.log('Contract deployed at:', deployed.deployTxData.contractAddress);
```

STEP 4 – CALL A CIRCUIT (PRIVATE TRANSAC

```
import { findDeployedContract }
  from '@midnight-ntwrk/midnight-js-contracts';

const contract = await findDeployedContract(providers, {
  contractAddress: deployed.deployTxData.contractAddress,
  compiledContract,
  privateStateId: 'my-state'
});

// Calling a circuit generates a ZK proof locally,
// then submits the transaction to the network.
const tx = await contract.callTx.increment();
await tx.wait();
\end{verbatim>}

\paragraph{Step 5 --- Query state.}

\begin{verbatim}
import { getPublicStates }
  from '@midnight-ntwrk/midnight-js-contracts';

const states = await getPublicStates(providers, contractAddress);
console.log('Current public state:', states);
```

Key Concepts to Understand

- **Circuits** — the functions you write in Compact (Midnight's language) that run locally and generate ZK proofs. Think of them as the private business logic.
- **Private state** — data stored on your device, encrypted with AES-256-GCM via LevelDB. It never appears on the blockchain.

- **Ledger state** — the public on-chain state of a contract. Anyone can read it; nobody can fake it.
- **ZK artifacts** — files produced by the Compact compiler: prover keys, verifier keys, and the ZKIR (Zero-Knowledge Intermediate Representation). These are loaded at runtime.
- **Transaction flow** — local execution of the circuit → proof generation by the proof server → balancing dust token fees → broadcast to the network.

Creating a Blockfrost Account

If you already have a Blockfrost account, Midnight support is included at no extra cost — just create a new project and select Midnight Mainnet. If you are new:

1. Sign up at `https://blockfrost.io` (free tier: 50,000 requests per day, no credit card required).
2. Click **Add Project** and select **Midnight Mainnet**.
3. Copy your API key and use it in the URL as shown above.

> Midnight requests share your existing Blockfrost rate limit. The GraphQL API also enforces per-query limits on depth, number of fields, complexity, and execution timeout. Requests that exceed these limits return an error describing which limit was hit.

For full SDK documentation see `docs.midnight.network/sdks/official/midnight-js` and for the Blockfrost-specific configuration see `docs.blockfrost.io/midnight/`.

ZK FOR COMPRESSION AND PRIVACY

4. PERFORMANCE, PRIVACY AND TRADE-OFFS

4.1. ZK FOR DATA MINIMISATION AND PROOF COMPRESSION

A point that is not stressed nearly enough: **ZK is not only about privacy**. In physics, light behaves as both a particle and a wave depending on how you observe it. ZK has a similarly dual nature — it can hide information, but it can also *compress* it. These two properties are completely independent, and in practice the compression use case is arguably more economically important today.

The Core Idea

Suppose an L2 network processes 100,000 transactions in one hour. Each transaction has a sender, a receiver, an amount, and a signature. If you posted all of that data to L1, you would overwhelm it and pay enormous fees.

The ZK alternative: the L2 executes all 100,000 transactions off-chain, then generates a single **ZK proof** attesting that every transaction was correctly signed by the legitimate owner of each wallet. The L1 smart contract verifies only the proof — a task that costs a fixed amount of gas regardless of how many transactions are bundled inside.

> **100,000 L2 transactions $\longrightarrow$ one ZK proof on L1.**
> The proof is a few kilobytes. The data it certifies could be gigabytes.

This is called a **ZK Rollup**. Nobody can lie about what happened: if the proof verifies, the state transition is correct. The blockchain does not need to see the individual transactions at all.

Why This Matters for Cardano

Consider a Cardano DEX. Today, every order executes on-chain: the validator checks amounts, validates prices, and enforces invariants for each UTxO — consuming script budget per order. With ZK the flow becomes:

1. The DEX batches many orders off-chain.
2. It generates a ZK proof that all orders were correctly matched and all balances balance.
3. A single on-chain transaction carries only the proof and the net state changes.

The on-chain validator verifies the proof. It does not re-execute the matching logic. This is orders of magnitude cheaper per order and allows throughput that would be impossible otherwise.

The Cost: Proof Generation Time

ZK proofs are not free to generate. The off-chain prover must run the circuit — which is an arithmetic encoding of your computation — over the full witness data. For large circuits this can take seconds to minutes on consumer hardware.

This is the fundamental trade-off of ZK compression: *the prover pays a large upfront computational cost so that the verifier pays almost nothing.* In a rollup setting the prover is a well-resourced sequencer, so the cost is amortised across all the transactions it bundles. For a single-user proof the user's device bears the cost.

4.2. SUCCINCTNESS AND SCALABILITY

The "S" in zk-SNARK stands for **Succinct**. A succinct proof has two properties:

- **Small size.** A Groth16 proof is ∼200 bytes regardless of the size of the computation it certifies. A PLONK proof is a few hundred bytes. A STARK proof is larger (∼50–200 KB) but still bounded.
- **Fast verification.** Verifying a proof takes milliseconds and a fixed amount of compute — again, regardless of the underlying computation size.

This asymmetry is what makes ZK interesting for blockchains. The blockchain is a slow, expensive, globally-replicated computer. You want to put as little work on it as possible. Succinctness lets you offload arbitrarily large computations and pay only a constant verification cost on-chain.

Recursive Proofs and Proof Aggregation

The compression story gets even more powerful with **recursive proofs**: a proof that another proof is valid. If you can verify a ZK proof inside a ZK circuit, you can build a tree of proofs:

- Each leaf proves a batch of transactions.
- Each internal node proves that its two children proofs are valid.
- The root is a single proof attesting to the entire tree.

This is how Mina compresses its entire blockchain history into 22 KB, and how recursive SNARKs like Nova allow continuous proof accumulation with near-zero overhead per step. The on-chain verifier always sees only one proof, regardless of how deep the tree is.

Scalability in Practice

Approach	Throughput	Security
Pure L1	Limited by block size	Native L1 security
Optimistic Rollup	10×–100×	L1 security + fraud proof delay
ZK Rollup	100×–1000×	L1 security + immediate finality
Recursive ZK	Near unlimited batching	Same as ZK Rollup

ZK Rollups have an important advantage over Optimistic Rollups: there is **no challenge window**. An Optimistic Rollup has to wait days before a withdrawal is final, to give time for fraud proofs. A ZK Rollup is final as soon as the proof verifies — typically within minutes of the batch being posted.

4.3. PRIVACY BY DESIGN: APPLICATIONS AND LIMITATIONS

Building with privacy is genuinely hard — not mathematically, but in terms of user experience.

When you build a standard DApp, you can see everything about your user: their balance, their history, their open positions. You can use this data to personalise the interface, warn them about risks, guide them through deposit flows, and show them relevant information. The transparency of the blockchain is, paradoxically, a *UX feature* for most applications.

The moment you introduce privacy, that transparency disappears. You can no longer read the user's balance. You cannot verify their eligibility client-side without the user handing you a proof. The interface has to work without the information it used to take for granted.

Tornado Cash as an Academic Case Study

> This book does not promote or endorse the use of Tornado Cash or any sanctioned platform. The following is a purely technical and UX analysis for educational purposes.

Tornado Cash is the clearest example of where ZK privacy meets UX friction. The flow for a user was:

1. Deposit funds into the contract. Receive a **note** — a file containing a cryptographic secret that proves ownership of the deposit.
2. Store the note securely (it is the only way to redeem; lose it and the funds are gone).
3. Later, from a different address with no connection to the original, present the note and a ZK proof that you know the secret. The contract releases the funds to a fresh address.

The cryptography is elegant. The UX is a disaster for non-technical users. Downloading

a file, storing it safely, transferring it across devices, and then running a separate withdrawal flow is completely alien to anyone who has not used a command-line wallet.

The cypherpunk community loves this kind of design — you are in full control, you hold your own keys and notes, nobody can censor you. But if the target audience is mass adoption, this is a hard wall. Most users will make a mistake, lose the note, or panic and try to contact support that does not exist.

Designing Private Interfaces

The lesson is not that privacy and good UX are incompatible — it is that building private applications requires re-thinking the interface from scratch rather than bolting ZK onto an existing transparent design. Some strategies that help:

- **Wallet-managed private state.** Instead of the user handling notes manually, the wallet generates and stores them automatically. Midnight's architecture does exactly this: private state lives encrypted in your local wallet, and the SDK manages proof generation behind the scenes.
- **Progressive disclosure.** Show the user only what they need to see at each step. The complexity of ZK proofs is an implementation detail — it does not belong in the UI.
- **Sensible defaults.** Make the private path the default rather than opt-in. Monero's design (always private) achieves better anonymity sets than Zcash's (opt-in) precisely because users do not have to make a choice.

4.4. THE TRADE-OFFS: PERFORMANCE VS. PRIVACY

Introducing ZK privacy fundamentally changes the computational model of your application, and it is worth being explicit about where the costs land.

Off-Chain Computation

Ethereum popularised an on-chain computation model: all logic executes on the network, results are public and globally verifiable, and the node is the source of truth. Privacy with ZK inverts this: **all meaningful computation happens off-chain**, and the on-chain component verifies only a proof that the computation was done correctly.

This shift has real consequences:

- **User devices bear the proving cost.** Generating a ZK proof can take seconds to minutes on a mobile device. This is improving rapidly with hardware acceleration and more efficient proof systems, but it is not free.
- **Application logic must be expressed as a circuit.** Not all computations are equally easy to express in arithmetic circuit form. Division, conditionals, and floating-point operations all require careful encoding. Languages like Noir, Circom, and Compact (Midnight) exist to make this easier, but there is still a learning curve.
- **Debugging is harder.** When a proof fails to verify, the error is usually a constraint violation deep inside the circuit — not a friendly stack trace. Tooling is improving but lags behind standard development environments.

Trusting a Proving Server

One workaround for slow on-device proving is to offload it to a **centralised proof server**. The user sends their inputs to the server, the server generates the proof, and returns it. This is faster, but it introduces a critical trust assumption: you are now telling a third party the very information you are trying to keep private.

> **The rule:** if the proving happens on a server you do not control, and that server can see your private inputs, then you do not have ZK privacy — you have *server-side secrecy*, which is a much weaker guarantee.

The correct architecture is to run the proof server locally. Midnight does this via a local Docker container: your private inputs go to a process on your own machine and never leave it. The tradeoff is that the user must run Docker, which is again a UX challenge.

Summary of Trade-offs

Choice	Benefit	Cost
On-device proving	True privacy	Slow, hardware-dependent
Centralised proof server	Fast, seamless UX	Privacy broken (trust the server)
ZK for compression only	Scalability, no UX friction	Complexity, proving time
Always-on privacy (Monero)	Large anonymity set	No opt-out, regulatory friction
Opt-in privacy (Zcash)	Regulatory flexibility	Small anonymity set

That is enough theory. From the next chapter onwards we will stop talking about ZK and start *building* it: our first ZK smart contract on Cardano using Plutus.

BUILDING ZK ON CARDANO: CIRCUITS AND VERIFICATION

5. CIRCOM, SNARKJS, AND AIKEN

5.1. THE CIRCUIT PIPELINE

Writing a ZK smart contract on Cardano is not a single-language problem. It involves a chain of tools that each play a different role:

1. **Circom** — a domain-specific language for describing arithmetic circuits.
2. **snarkjs** — a JavaScript library that runs the trusted setup ceremony, compiles witness calculations, and generates / verifies Groth16 proofs.
3. **Aiken** — the on-chain language where the Cardano validator lives and where the ZK proof is verified against the circuit's public verification key.

The high-level flow is:

$$\begin{array}{ccccc} \text{Circom circuit} & \longrightarrow & \text{snarkjs trusted setup} & \longrightarrow & \text{verification key} \\ & & \downarrow & & \downarrow \\ & & \text{Groth16 proof} & \longrightarrow & \text{Aiken validator} \end{array}$$

The circuit defines the *rules*. The trusted setup ceremony produces the public parameters that let anyone prove and verify statements about those rules. The proof is generated off-chain by the user. The Aiken validator checks the proof on-chain — a single, fast pairing check using the BLS12-381 primitives introduced in Plutus V3.

5.2. WRITING A CIRCUIT IN CIRCOM

Circom is a language for expressing computations as **arithmetic circuits** — networks of addition and multiplication gates over a finite field. Every statement you want to prove must be reducible to a set of polynomial constraints called an **R1CS** (Rank-1 Constraint System).

Installing the Tools

Install snarkjs globally:

```
npm install -g snarkjs
```

Install Circom by following the official guide at `docs.circom.io/getting-started/installation`. On Linux/macOS this compiles from Rust source:

```
git clone https://github.com/iden3/circom.git
cd circom
cargo build --release
cargo install --path circom
```

Install the circomlib circuit library (contains Poseidon and other primitives):

```
npm install -D circomlib
```

The Janus Wallet Authentication Circuit

Janus Wallet[1] is the most complete open-source ZK project on Cardano. It uses a circuit that proves a user knows their password without ever sending it on-chain. The circuit is built on the **Poseidon** hash function — a ZK-friendly hash designed to be cheap to prove inside arithmetic circuits (unlike SHA-256, which is extremely expensive to circuit-encode).

The circuit has five **public** inputs (visible on-chain) and one **private** input (the secret that never leaves the user's device):

```
pragma circom 2.1.6;

include "../../../node_modules/circomlib/circuits/poseidon.circom";

template Authenticate() {
    signal input userId;        // Public: username identifier
    signal input credentialHash; // Public: Poseidon(pwd, userId)
    signal input challenge;      // Public: nonce from the smart contract
    signal input challengeFlag;  // Public: overflow flag for challenge
    signal input circuitHash;    // Public: Poseidon(pwd, userId, credentialHash,
                                 //                  challenge, challengeFlag)
    signal input pwd;            // Private: user password

    // Step 1: verify credentialHash = Poseidon(pwd, userId)
    component p = Poseidon(2);
    p.inputs[0] <== pwd;
    p.inputs[1] <== userId;
    credentialHash === p.out;

    // Step 2: verify circuitHash = Poseidon(pwd, userId, credentialHash,
    //                                       challenge, challengeFlag)
    component poseidon = Poseidon(5);
    poseidon.inputs[0] <== pwd;
    poseidon.inputs[1] <== userId;
    poseidon.inputs[2] <== credentialHash;
    poseidon.inputs[3] <== challenge;
    poseidon.inputs[4] <== challengeFlag;
    circuitHash === poseidon.out;
}

component main {public [userId, credentialHash, challenge,
                        challengeFlag, circuitHash]} = Authenticate();
```

The `===` operator asserts a constraint: if the constraint is violated, no valid proof can be produced. The `<==` operator assigns and constrains in one step.

[1] https://github.com/leobel/janus-wallet

The `challenge` and `challengeFlag` fields are critical for replay protection. The smart contract changes the challenge after every successful spend, so a proof generated for one transaction cannot be reused in another.

Compiling the Circuit

```
circom --r1cs --wasm --c --sym --inspect \
       --prime bls12381 circuit.circom -o .
```

> Run this command from the directory where your `include` path can reach the circomlib node module. For Janus Wallet this is `src/zkproof/janus-wallet/`.

The flags produce:

- `circuit.r1cs` — the constraint system in binary format.
- `circuit_js/circuit.wasm` — WebAssembly witness calculator.
- `circuit.sym` — symbol table for debugging.

The `--prime bls12381` flag targets the BLS12-381 curve, which matches Cardano's on-chain primitives.

Inspect the compiled circuit:

```
snarkjs r1cs info circuit.r1cs
```

For the Janus circuit you should see:

```
[INFO] snarkJS: Curve: bls12-381
[INFO] snarkJS: # of Wires: 1357
[INFO] snarkJS: # of Constraints: 1352
[INFO] snarkJS: # of Private Inputs: 1
[INFO] snarkJS: # of Public Inputs: 5
[INFO] snarkJS: # of Outputs: 0
```

5.3. THE TRUSTED SETUP CEREMONY

Groth16 requires a two-phase trusted setup. Phase 1 (Powers of Tau) is circuit-independent and can be reused across projects. Phase 2 is circuit-specific and ties the parameters to your exact R1CS.

Phase 1: Powers of Tau

1. START A NEW CEREMONY.

```
snarkjs powersoftau new bls12381 14 pot14_0000.ptau -v
```

The second argument (14) is the power of two of the maximum number of constraints: $2^{14} = 16{,}384$. Choose the smallest power that fits your circuit to keep file sizes manageable. The maximum supported is $2^{28} \approx 268$ million constraints.

2-3. CONTRIBUTE RANDOMNESS.

Multiple contributors make the ceremony secure: the parameters are safe as long as *at least one* contributor honestly discarded their secret after contributing.

```
snarkjs powersoftau contribute pot14_0000.ptau pot14_0001.ptau \
    --name="First contribution" -v

snarkjs powersoftau contribute pot14_0001.ptau pot14_0002.ptau \
    --name="Second contribution" -v -e="some random text"
```

The `-e` flag provides entropy inline, making the step non-interactive — useful in scripts.

4. (OPTIONAL) THIRD-PARTY CONTRIBUTION.

You can mix contributions from different software:

```
snarkjs powersoftau export challenge pot14_0002.ptau challenge_0003

snarkjs powersoftau challenge contribute bls12381 challenge_0003 \
    response_0003 -e="some random text"

snarkjs powersoftau import response pot14_0002.ptau response_0003 \
    pot14_0003.ptau -n="Third contribution name"
```

5. VERIFY THE CEREMONY SO FAR.

```
snarkjs powersoftau verify pot14_0003.ptau
```

A clean run ends with `Powers Of tau OK!`

6. APPLY A RANDOM BEACON.

A random beacon finalises Phase 1. It is a delayed hash function evaluated on a publicly known, high-entropy value — for example, the hash of a specific Cardano block, or a national lottery result announced after the ceremony began.

```
snarkjs powersoftau beacon pot14_0003.ptau pot14_beacon.ptau \
    3c692631872308ff1f9fed102b940aecee160def9c2478b50b977091 \
    10 -n="Final Beacon"
```

The `10` argument means $2^{10} = 1024$ iterations of the hash — enough to make the beacon computationally irreversible.

> **Why a random beacon?** If a contributor could predict the beacon before their contribution, they might be able to bias the parameters. An unpredictable, publicly verifiable source of randomness prevents this. Janus Wallet uses the `policy_id` under which all account tokens are minted as its beacon value.

7. PREPARE PHASE 2.

```
snarkjs powersoftau prepare phase2 pot14_beacon.ptau pot14_final.ptau -v
```

This pre-computes the encrypted Lagrange polynomial evaluations needed in Phase 2.

8. FINAL VERIFICATION.

```
snarkjs powersoftau verify pot14_final.ptau
```

Unlike step 5, this run will *not* warn about missing phase 2 values — that means the file is ready.

Phase 2: Circuit-Specific Setup

9-10. COMPILE THE CIRCUIT AND GENERATE T

```
snarkjs groth16 setup circuit.r1cs pot14_final.ptau circuit_0000.zkey
```

> Do not use `circuit_0000.zkey` in production. It has zero contributions and is not safe.

11-12. ADD CONTRIBUTIONS TO PHASE 2.

```
snarkjs zkey contribute circuit_0000.zkey circuit_0001.zkey \
    --name="1st Contributor Name" -v

snarkjs zkey contribute circuit_0001.zkey circuit_0002.zkey \
    --name="Second contribution" -v -e="Another random entropy"
```

13. (OPTIONAL) THIRD-PARTY PHASE 2 CONTR

```
snarkjs zkey export bellman circuit_0002.zkey challenge_phase2_0003
snarkjs zkey bellman contribute bls12381 challenge_phase2_0003 \
    response_phase2_0003 -e="some random text"
snarkjs zkey import bellman circuit_0002.zkey response_phase2_0003 \
    circuit_0003.zkey -n="Third contribution name"
```

14. VERIFY THE ZKEY.

```
snarkjs zkey verify circuit.r1cs pot14_final.ptau circuit_0003.zkey
```

Output: `[INFO] snarkJS: ZKey Ok!`

15. APPLY THE RANDOM BEACON AND EXPORT T

```
snarkjs zkey beacon circuit_0003.zkey circuit_final.zkey \
    0102030405060708090a0b0c0d0e0f101112131415161718191a1b1c1d1e1f \
    10 -n="Final Beacon phase2"

snarkjs zkey verify circuit.r1cs pot14_final.ptau circuit_final.zkey

snarkjs zkey export verificationkey circuit_final.zkey verification_key.json
```

`verification_key.json` contains the public parameters that will be embedded in the Aiken smart contract (as an on-chain token) and used to verify every proof.

5.4. GENERATING AND VERIFYING A PROOF

Prepare Inputs

Create an `input.json` with the circuit's signals. Note that large integers must be quoted strings to avoid JSON precision loss:

```
{
    "userId": "52435875175126190479447740508185965837690552500527637822...",
    "credentialHash": "17469782945530594987388377835453341901430155623074...",
    "challenge": "17469782945530594987388377835453341901430155623074104...",
    "challengeFlag": "1",
    "circuitHash": "24883872159189864133794802823127472657146334491119345...",
    "pwd": "52435875175126190479447740508185965837690552500527637822..."
}
```

Calculate the Witness

The witness is the complete set of wire values for all signals in the circuit given your specific inputs.

```
snarkjs wtns calculate circuit_js/circuit.wasm input.json witness.wtns
snarkjs wtns check circuit.r1cs witness.wtns
```

A valid witness prints `WITNESS IS CORRECT`.

Generate the Proof

```
snarkjs groth16 prove circuit_final.zkey witness.wtns proof.json public.json
```

This produces:

- `proof.json` — the Groth16 proof (π_A, π_B, π_C elliptic curve points).
- `public.json` — the values of the public inputs (visible to the verifier).

Or in a single step:

```
snarkjs groth16 fullprove input.json circuit_js/circuit.wasm \
    circuit_final.zkey proof.json public.json
```

Verify Off-Chain

```
snarkjs groth16 verify verification_key.json public.json proof.json
```

Output: `[INFO] snarkJS: OK!`

5.5. PREPARING THE VERIFICATION KEY FOR CARDANO

Cardano's BLS12-381 primitives work exclusively with **compressed** elliptic curve points. The `verification_key.json` produced by snarkjs uses *uncompressed* points. Before moving on-chain you must compress all $\mathbb{G}_1$ and $\mathbb{G}_2$ points: `vk_alpha_1`, `vk_beta_2`, `vk_gamma_2`, `vk_delta_2`, and every point in the `IC` array.

Janus Wallet provides a script for this:

```
node --no-warnings --experimental-specifier-resolution=node \
     --loader ts-node/esm compress_zk_verification_key.ts \
     janus-wallet/verification_key.json \
     janus-wallet/compressed_verification_key.json
```

Run this from `src/zkproof`. The output `compressed_verification_key.json` is what goes on-chain.

5.6. ON-CHAIN VERIFICATION IN AIKEN

The verification key is minted as a token on Cardano using Janus Wallet's circuit minting endpoint. This makes the parameters immutable and publicly auditable — anyone can inspect the token and verify the ceremony.

Minting the Circuit Token

```
POST http://localhost:3001/circuits
{
    "token_name": "Circuit#000",
    "version": 0,
    "nonce": "some random value",
    "circuit": {
        "vk_alpha1":  "8f944ea2...",
        "vk_beta2":   "81dfe069...",
        "vk_gamma2":  "93e02b60...",
        "vk_delta2":  "910d1b3d...",
        "vk_ic": [
            "8d043ff5...",
            "86fa74a9...",
            ...
        ]
    }
}
```

The endpoint mints the token under a `policy_id` derived from the nonce. The nonce can be the beacon value used in Phase 2, linking the on-chain token cryptographically to the ceremony transcript.

How the Aiken Validator Works

The Aiken validator performs a Groth16 pairing check using the BLS12-381 primitives:

1. It reads the compressed verification key from the circuit token in the reference input.
2. It reads the proof (π_A, π_B, π_C) from the redeemer.

3. It reads the public inputs (userId, credentialHash, challenge, challengeFlag, circuitHash) from the datum.
4. It computes the pairing equation and calls `final_verify` to check the proof.
5. If the proof is valid, the UTxO is unlocked. If not, the transaction fails.

The Aiken test suite from Janus Wallet shows all passing scenarios:

```
aiken check -t silent .

+- tests/verify ----------------------------------------------
| PASS [mem:  89791, cpu: 2541340679] verify_zk_proof
| PASS [mem:  89791, cpu: 2541340679] verify_zk_proof_1
| PASS [mem:  63964, cpu: 1355803465] verify_zk_proof_invalid_user
| PASS [mem:  63964, cpu: 1355812874] verify_zk_proof_invalid_circuit_hash
| PASS [mem:  63964, cpu: 1355812874] verify_zk_proof_invalid_challenge
| PASS [mem:  63964, cpu: 1355812874] verify_zk_proof_invalid_proof
+---------------------------------------- 11 tests | 11 passed | 0 failed
```

Notice the CPU budget for `verify_zk_proof`: approximately 2.5 billion units. Groth16 verification on BLS12-381 is one of the most expensive operations you can perform on-chain, but it is within Cardano's per-transaction budget — which is why Plutus V3 added these primitives in the first place.

5.7. JANUS WALLET: PUTTING IT ALL TOGETHER

Janus Wallet (`https://github.com/leobel/janus-wallet`) is the reference implementation of a ZK smart contract wallet on Cardano. Instead of signing transactions with a private key, the user proves knowledge of their password via a Groth16 circuit. The private key never exists — there is nothing to lose or steal.

Architecture

- **Circuit** — the Circom Authenticate template described above.
- **Off-chain server** — a Node.js HTTP API that builds, signs (via ZK proof), and submits transactions.
- **On-chain validator** — an Aiken smart contract that verifies the Groth16 proof against the circuit token.
- **PostgreSQL** — stores user accounts, circuit data, and UTxO state.

Running the Server

Create a `.env` file:

```
PORT=3001
DB_HOST=localhost
DB_PORT=5432
DB_USERNAME=janus
```

```
DB_PASSWORD=123456
DB_NAME=janus_wallet
CARDANO_NETWORK=Preview
BLOCKFROST_API_URL=https://cardano-preview.blockfrost.io/api/v0
BLOCKFROST_API_KEY=preview...
WALLET_PRV_KEY=ed25519_sk...
COLLATERAL_PRV_KEY=ed25519_sk...
COLLATERAL_ADDRESS=addr_test...
```

Run database migrations and start the server:

```
npm run migrate
npm start
```

> The signing logic that produces the ZK proof depends on `circom` being accessible from the process running the server. Verify with `circom --version`. If the binary is at `$HOME/.cargo/bin` and not on your PATH, copy it to `/usr/local/bin`.

User Lifecycle

REGISTER A USER.

```
POST http://localhost:3001/wallets/{user_name}
{
    "hash":     "42b79954...",  // Poseidon(password, username)
    "kdf_hash": "243262..."     // bcrypt(password, salt)
}
```

The server generates a smart contract address unique to this user and mints a profile NFT to it. The `hash` is the on-chain credential; the `kdf_hash` lets the server authenticate requests without storing the plaintext password.

SEND FUNDS (THREE STEPS).

```
// 1. Build the unsigned transaction
POST http://localhost:3001/wallets/{user_id}/build
{ "amount": 5000000, "receive_address": "addr_test1..." }
// Response: { "tx": "84a900..." }

// 2. Sign with the ZK proof
POST http://localhost:3001/wallets/{user_id}/sign
{ "pwd": "supersecret", "tx": "84a900..." }
```

```
// 3. Submit
POST http://localhost:3001/wallets/{user_id}/send
{
    "redeemers": ["d8799f..."],  // ZK proof as CBOR
    "tx": "84a900..."
}
```

The `redeemers` array contains the Groth16 proof serialised as CBOR. Additional redeemers in the array are bypass markers for UTxOs from the same wallet that do not require a fresh proof.

STAKE, DELEGATE, AND WITHDRAW.

Janus Wallet supports the full Cardano staking lifecycle through the same three-step pattern (build → sign → send):

```
POST /wallets/{user_id}/pools/{pool_id}/registerAndDelegate
POST /wallets/{user_id}/pools/{pool_id}/delegate
POST /wallets/{user_id}/dreps/{drep_id}/delegate
POST /wallets/{user_id}/withdraw  { "amount": 1234567 }
```

> **The key insight:** from the blockchain's perspective, every Janus transaction is a normal smart contract spend. The only thing that makes it a ZK wallet is the redeemer — a Groth16 proof that the user knows their password, checked on-chain by the Aiken validator in a single pairing verification.

Full Test Results

Janus Wallet ships with 29 Aiken tests covering minting, spending, challenge construction, and ZK verification — all passing with zero errors:

```
Summary 29 checks, 0 errors, 19 warnings
```

The warnings are informational (unused variables in test helpers) and do not affect correctness.

This is the complete picture of ZK on Cardano today: Circom defines the computation, snarkjs runs the ceremony and generates proofs, and Aiken verifies them on-chain using native BLS12-381 pairing operations. In the next chapters we will build our own circuit and validator from scratch.

YOUR FIRST ZK CONTRAC
ON CARDANO

6. PASSWORD-LOCKED UTXO WITH GROTH16

6.1. WHAT WE ARE BUILDING

This chapter builds the simplest possible ZK smart contract on Cardano: a UTxO that can only be unlocked by someone who can *prove* they know a secret password — without ever revealing the password on-chain.

The password is hashed with **Poseidon**, a ZK-friendly hash function. The hash is stored publicly in the datum. To spend the UTxO, the spender generates a Groth16 proof that they know a pre-image of that hash. The Aiken validator verifies the proof on-chain using the BLS12-381 pairing primitives introduced in Plutus V3.

> **What stays private:** the password. **What goes on-chain:** the Poseidon hash, the verification key, and the proof.

The complete source lives in `zk-password/` at the root of the repository. By the end of this chapter the directory will look like this:

```
zk-password/
+-- aiken.toml
+-- package.json
+-- circuit/
|   +-- circuit.circom          <- Circom constraint circuit
|   +-- poseidon_hash.circom    <- helper: compute hash without constraint
|   +-- gen_input.js            <- script: password -> input.json
|   +-- compress_vk.js          <- script: compress verification key
|   +-- compress_proof.js       <- script: compress proof points
+-- lib/
|   +-- zk_password/
|       +-- groth16.ak          <- Groth16 verifier library
|       +-- password_tests.ak   <- Aiken unit tests
+-- validators/
    +-- password.ak             <- on-chain validator
```

6.2. PROJECT SETUP

Create the Aiken project and initialise npm for the JavaScript utility scripts:

```
aiken new chapter6/zk-password
cd zk-password
npm init -y
npm install -D circomlib
npm install @noble/curves
```

`circomlib` provides the Poseidon circuit templates. `@noble/curves` is used by the compression scripts to convert uncompressed BLS12-381 points to the 48-byte (G1) and

96-byte (G2) format that Cardano accepts.

6.3. THE CIRCUIT

Create `circuit/circuit.circom`:

```
pragma circom 2.1.6;

include "../node_modules/circomlib/circuits/poseidon.circom";

// Prove knowledge of a password whose Poseidon hash matches a commitment.
//
// Public inputs  (visible on-chain in the datum):
//   hash  -- Poseidon(pwd)
//
// Private inputs (never leave the prover's device):
//   pwd   -- the password
template PasswordLock() {
    signal input hash;   // public
    signal input pwd;    // private

    component h = Poseidon(1);
    h.inputs[0] <== pwd;

    hash === h.out;
}

component main {public [hash]} = PasswordLock();
```

The `===` operator defines an arithmetic constraint: if the Poseidon hash of `pwd` does not equal `hash`, the constraint system is unsatisfiable and no valid proof can ever be constructed. The `<==` operator assigns a value and simultaneously constrains the signal.

Compile the Circuit

From inside `zk-password/`:

```
circom --r1cs --wasm --sym --inspect \
       --prime bls12381 circuit/circuit.circom -o circuit/
```

The `--prime bls12381` flag targets the BLS12-381 scalar field, which matches Cardano's on-chain pairing primitives. The outputs land in `circuit/`:

- `circuit.r1cs` — the binary constraint system.
- `circuit_js/circuit.wasm` — the WebAssembly witness calculator.
- `circuit.sym` — the symbol table (signal names and witness indices).

Inspect the compiled R1CS:

```
template instances: 71
non-linear constraints: 216
linear constraints: 199
public inputs: 1
private inputs: 1
public outputs: 0
wires: 417
labels: 583
Written successfully: circuit/circuit.r1cs
Written successfully: circuit/circuit.sym
Written successfully: circuit/circuit_js\circuit.wasm
Everything went okay
```

Circom compilation output. Our circuit has 216 non-linear constraints, 1 public input (`hash`), and 1 private input (`pwd`).

```
snarkjs r1cs info circuit/circuit.r1cs
```

You should see `Curve: bls12-381`, 1 public input, 1 private input, and roughly 240 constraints (mostly Poseidon round internals).

6.4. GENERATING INPUT.JSON

The circuit needs two inputs: the password as a field element and its Poseidon hash. There is a subtlety here worth understanding before writing any code.

The Field-Element Problem

The `pwd` signal in the circuit is a BLS12-381 field element — an integer, not a string. A plain numeric string like `"12345"` converts trivially with `BigInt("12345")`. A word like `"mypassword"` does not, and calling `BigInt("mypassword")` throws a runtime error.

The solution is to encode the string as a big-endian integer of its UTF-8 bytes:

```
function toFieldElement(s) {
  if (/^\d+$/.test(s)) return BigInt(s);
  return BigInt("0x" + Buffer.from(s, "utf8").toString("hex"));
}
```

The Hash Computation Problem

We also need the Poseidon hash of the password, but we cannot just call `circomlibjs`'s `buildPoseidon` here. That library implements Poseidon over the **BN128** field; our circuit uses the **BLS12-381** scalar field. The two fields have different primes, so the hash values differ.

We solve this by compiling a tiny helper circuit that simply exposes the hash as an output signal — no equality constraint, just a wire:

`circuit/poseidon_hash.circom`:

```
pragma circom 2.1.6;

include "../node_modules/circomlib/circuits/poseidon.circom";

template HashOnly() {
    signal input  pwd;
    signal output hash;
    component h = Poseidon(1);
    h.inputs[0] <== pwd;
    hash <== h.out;
}

component main = HashOnly();
```

Compile it with the same BLS12-381 prime:

```
circom --wasm --prime bls12381 \
       circuit/poseidon_hash.circom -o circuit/
```

Because this circuit has no `===` constraint, the witness calculator runs cleanly for any input and `witness[1]` — the first public output — contains `Poseidon(pwd)` computed over the correct field.

circuit/gen_input.js

```
const { readFileSync, writeFileSync } = require("fs");
const path = require("path");

const password = process.argv[2] ?? "12345";

function toFieldElement(s) {
  if (/^\d+$/.test(s)) return BigInt(s);
  return BigInt("0x" + Buffer.from(s, "utf8").toString("hex"));
}

const pwd = toFieldElement(password);

async function main() {
  const buildWC = require("./poseidon_hash_js/witness_calculator.js");
  const wasm = readFileSync(
    path.join(__dirname, "poseidon_hash_js", "poseidon_hash.wasm")
  );
  const wc = await buildWC(wasm);

  const witness = await wc.calculateWitness({ pwd }, true);
  const hash = witness[1]; // output signal is always at index 1

  const input = { hash: hash.toString(), pwd: pwd.toString() };
  const outPath = path.join(__dirname, "input.json");
  writeFileSync(outPath, JSON.stringify(input, null, 2));
```

```
  console.log("password :", password);
  console.log("hash     :", hash.toString());
  console.log("written  :", outPath);
}

main().catch(err => { console.error(err); process.exit(1); });
```

Usage:

```
node circuit/gen_input.js 123456
node circuit/gen_input.js mypassword
node circuit/gen_input.js "hello world"
```

The resulting `circuit/input.json` contains the hash and password as quoted decimal strings, ready for `snarkjs groth16 fullprove`.

6.5. THE TRUSTED SETUP

Groth16 requires a two-phase trusted setup. Phase 1 (Powers of Tau) is circuit-independent; Phase 2 is specific to your R1CS.

Phase 1 – Powers of Tau

```
snarkjs powersoftau new bls12381 12 pot12_0000.ptau -v
snarkjs powersoftau contribute pot12_0000.ptau pot12_0001.ptau \
    --name="dev" -e="some entropy"
snarkjs powersoftau beacon pot12_0001.ptau pot12_beacon.ptau \
    0000000000000000000000000000000000 10 -n="Dev beacon"
snarkjs powersoftau prepare phase2 pot12_beacon.ptau pot12_final.ptau -v
```

We use $2^{12} = 4096$ max constraints — well above our 216-constraint circuit and fast to generate. Use a larger power (e.g. 14) for bigger circuits.

Phase 2 – Circuit-Specific Setup

```
snarkjs groth16 setup circuit/circuit.r1cs pot12_final.ptau zkey_0000.zkey
snarkjs zkey contribute zkey_0000.zkey zkey_0001.zkey \
    --name="dev" -e="more entropy"
snarkjs zkey beacon zkey_0001.zkey zkey_final.zkey \
    0000000000000000000000000000000000 10 -n="Final beacon"
snarkjs zkey export verificationkey zkey_final.zkey verification_key.json
```

> For production you should use an existing BLS12-381 Powers of Tau file (e.g. from the Zcash ceremony) and run at least three independent Phase 2 contributions. The parameters here are safe only for local development.

```
[INFO]  snarkJS: Circuit Hash:
                53b173e7 5f0fa4a9 cde8c117 4812a0a0
                7ff9248a 2763bba1 03a792f5 8eac5f2f
                84d032d4 fdae9a37 0c32f55b ac123c23
                bf0b1461 b1ac49ec b0b9d223 1dec1dfe
[INFO]  snarkJS: Contribution Hash:
                b648e692 31a53b0c 0322b310 c4619b65
                c6bce647 31d34b08 ba492013 c75de9d2
                c2602285 1684b22c d7d37f6d 82f5cb68
                bf57d0c7 60c11f28 85343355 0c5673fc
PS C:\Users\raulr\Documents\GitHub\ZK-from-zero-on-Cardano\zk-password> sn
 -n="Final beacon"
[INFO]  snarkJS: Contribution Hash:
                0fed237b ed8f4342 064d246a d5add71b
                f4de0314 61fba467 14aafec8 fa826e31
                fd46cf7b e2a98810 cb615e83 56561d83
                55c7f100 cbeff94d f2c7db3b f37ff1a4
PS C:\Users\raulr\Documents\GitHub\ZK-from-zero-on-Cardano\zk-password> sn
[INFO]  snarkJS: EXPORT VERIFICATION KEY STARTED
[INFO]  snarkJS: > Detected protocol: groth16
[INFO]  snarkJS: EXPORT VERIFICATION KEY FINISHED
```

snarkjs exports the verification key to `verification_key.json`. This file contains the uncompressed BLS12-381 G1 and G2 points.

6.6. GENERATING THE PROOF

With `circuit/input.json`, the compiled WASM, and the final zkey, generate the proof in one command:

```
snarkjs groth16 fullprove \
    circuit/input.json \
    circuit/circuit_js/circuit.wasm \
    zkey_final.zkey \
    proof.json public.json
```

Verify the proof off-chain before going further:

```
snarkjs groth16 verify verification_key.json public.json proof.json
```

You should see `[INFO] snarkJS: OK!`. The `public.json` file contains the single public input: the Poseidon hash of your password.

6.7. COMPRESSING POINTS FOR CARDANO

Cardano's BLS12-381 builtins only accept **compressed** elliptic-curve points. The snarkjs output uses uncompressed Weierstrass coordinates $[x, y, 1]$ for G1 and $[[x_0, x_1], [y_0, y_1], [1, 0]]$ for G2. Compressed form encodes a point in its x-coordinate alone (48 bytes for G1, 96 bytes for G2), using one flag bit to disambiguate which of the two possible y values to use.

We compress both the verification key and the proof with the `@noble/curves` library.

circuit/compress_vk.js

```
const { readFileSync, writeFileSync } = require("fs");
const path = require("path");

const vkPath  = process.argv[2]
    ?? path.join(__dirname, "..", "verification_key.json");
const outPath = process.argv[3]
    ?? path.join(__dirname, "..", "compressed_vk.json");

async function main() {
  const { bls12_381 } = await import("@noble/curves/bls12-381.js");
  const G1 = bls12_381.G1.Point;
  const G2 = bls12_381.G2.Point;

  const vk = JSON.parse(readFileSync(vkPath, "utf8"));

  function compressG1([x, y]) {
    return Buffer.from(
      G1.fromAffine({ x: BigInt(x), y: BigInt(y) }).toBytes(true)
    ).toString("hex");
  }

  function compressG2([[x0, x1], [y0, y1]]) {
    return Buffer.from(
      G2.fromAffine({
        x: { c0: BigInt(x0), c1: BigInt(x1) },
        y: { c0: BigInt(y0), c1: BigInt(y1) },
      }).toBytes(true)
    ).toString("hex");
  }

  const compressed = {
    vk_alpha1: compressG1(vk.vk_alpha_1),
    vk_beta2:  compressG2(vk.vk_beta_2),
    vk_gamma2: compressG2(vk.vk_gamma_2),
    vk_delta2: compressG2(vk.vk_delta_2),
    vk_ic:     vk.IC.map(compressG1),
  };

  writeFileSync(outPath, JSON.stringify(compressed, null, 2));
}

main().catch(err => { console.error(err); process.exit(1); });
```

```
node circuit/compress_vk.js
```

The resulting `compressed_vk.json` has:

- `vk_alpha1` — 96 hex chars (48 bytes, compressed G1)
- `vk_beta2`, `vk_gamma2`, `vk_delta2` — 192 hex chars each (96 bytes, compressed G2)
- `vk_ic` — array of two 96-hex-char strings; the first element is the constant term IC_0, the second is IC_1 (one entry per public input)

circuit/compress_proof.js

The proof points use the same approach:

```
const { readFileSync, writeFileSync } = require("fs");
const path = require("path");

const proofPath = process.argv[2]
    ?? path.join(__dirname, "..", "proof.json");
const outPath   = process.argv[3]
    ?? path.join(__dirname, "..", "compressed_proof.json");

async function main() {
  const { bls12_381 } = await import("@noble/curves/bls12-381.js");
  const G1 = bls12_381.G1.Point;
  const G2 = bls12_381.G2.Point;

  const proof = JSON.parse(readFileSync(proofPath, "utf8"));

  const compressed = {
    pi_a: Buffer.from(
      G1.fromAffine({
        x: BigInt(proof.pi_a[0]),
        y: BigInt(proof.pi_a[1]),
      }).toBytes(true)
    ).toString("hex"),
    pi_b: Buffer.from(
      G2.fromAffine({
        x: { c0: BigInt(proof.pi_b[0][0]), c1: BigInt(proof.pi_b[0][1]) },
        y: { c0: BigInt(proof.pi_b[1][0]), c1: BigInt(proof.pi_b[1][1]) },
      }).toBytes(true)
    ).toString("hex"),
    pi_c: Buffer.from(
      G1.fromAffine({
        x: BigInt(proof.pi_c[0]),
        y: BigInt(proof.pi_c[1]),
      }).toBytes(true)
    ).toString("hex"),
  };

  writeFileSync(outPath, JSON.stringify(compressed, null, 2));
}

main().catch(err => { console.error(err); process.exit(1); });
```

```
node circuit/compress_proof.js
```

6.8. THE GROTH16 VERIFIER LIBRARY

The file `lib/zk_password/groth16.ak` is a self-contained Groth16 verifier modelled after Janus Wallet's implementation.

Types

```
pub type VerificationKey {
  alpha: ByteArray,        // G1 - 48 bytes
  beta:  ByteArray,        // G2 - 96 bytes
  gamma: ByteArray,        // G2 - 96 bytes
  delta: ByteArray,        // G2 - 96 bytes
  ic:    List<ByteArray>,  // G1 points; length = num_public_inputs + 1
}

pub type Proof {
  pi_a: ByteArray,  // G1 - 48 bytes
  pi_b: ByteArray,  // G2 - 96 bytes
  pi_c: ByteArray,  // G1 - 48 bytes
}
```

All fields are compressed bytes. The `ic` list has two elements for a circuit with one public input: the constant term IC_0 and the coefficient IC_1 for `hash`.

The Verification Function

The pairing equation that Groth16 checks is:

$$e(\pi_A, \pi_B) = e(\alpha, \beta) \cdot e(\mathbf{vk_x}, \gamma) \cdot e(\pi_C, \delta)$$

where $\mathbf{vk_x} = IC_0 + \sum_i (\text{public_input}_i \cdot IC_{i+1})$.

```
pub fn groth16_verify(
  vk: VerificationKey,
  proof: Proof,
  public_inputs: List<Scalar>,
) -> Bool {
  let a = g1.decompress(proof.pi_a)
  let b = g2.decompress(proof.pi_b)
  let c = g1.decompress(proof.pi_c)

  let vk_x =
    when vk.ic is {
      [] -> fail @"vk.ic must not be empty"
      [ic0, ..rest] -> derive(rest, public_inputs, g1.decompress(ic0))
    }

  let e_ab         = bls12_381_miller_loop(a, b)
  let e_alpha_beta =
    bls12_381_miller_loop(g1.decompress(vk.alpha), g2.decompress(vk.beta))
  let e_x_gamma    = bls12_381_miller_loop(vk_x, g2.decompress(vk.gamma))
  let e_c_delta    = bls12_381_miller_loop(c, g2.decompress(vk.delta))
```

```
  let rhs =
    bls12_381_mul_miller_loop_result(
      bls12_381_mul_miller_loop_result(e_alpha_beta, e_x_gamma),
      e_c_delta,
    )

  bls12_381_final_verify(e_ab, rhs)
}
```

The entire cryptographic work happens inside Cardano's builtin functions (`bls12_381_miller_loop` and `bls12_381_final_verify`). These are native operations in Plutus V3 and are priced at fixed, low execution units regardless of proof complexity.

6.9. THE AIKEN VALIDATOR

The Opaque Scalar Problem

A natural first attempt for the datum would be:

```
pub type Datum {
  credential_hash: Scalar,  -- would be natural, but WRONG
  vk: VerificationKey,
}
```

The Aiken compiler immediately rejects this:

```
x I caught an opaque type trying to escape
  |
  | datum: Option<Datum>,
  |                  invalid validator's boundary
  |
  = Opaque types cannot figure anywhere in an outward-facing type
    like a validator's redeemer or datum.
```

`Scalar` is *opaque*: its constructor is hidden and it enforces invariants that cannot be expressed as plain Plutus Data. Opaque types cannot cross the on-chain/off-chain boundary (datum or redeemer), because the off-chain world has no way to build them safely from raw bytes.

The fix: store the hash as a plain `Int` in the datum (a normal Plutus integer that any off-chain tool can construct), and convert it to a `Scalar` at runtime inside the validator body using `scalar.new`:

```
use aiken/crypto/bls12_381/scalar
use cardano/transaction.{OutputReference, Transaction}
use zk_password/groth16.{Proof, VerificationKey, groth16_verify}
```

```
pub type Datum {
  credential_hash: Int,   // Poseidon(pwd) as a plain integer
  vk: VerificationKey,    // compressed Groth16 verification key
}

pub type Redeemer {
  proof: Proof,           // compressed Groth16 proof
}

validator password_lock {
  spend(
    datum: Option<Datum>,
    redeemer: Redeemer,
    _self: OutputReference,
    _ctx: Transaction,
  ) {
    expect Some(d) = datum
    // Convert Int -> Scalar at runtime. scalar.new returns None if
    // the value is outside [0, field_prime), which rejects malformed datums.
    expect Some(hash_scalar) = scalar.new(d.credential_hash)
    groth16_verify(d.vk, redeemer.proof, [hash_scalar])
  }

  else(_) {
    False
  }
}
```

The datum stores two things: the Poseidon hash of the password (the lock) and the verification key (the lock's mechanism). Both are public. The redeemer provides the proof (the key). The key never reveals the password.

Compile

```
aiken check
```

You should see zero errors and zero warnings.

6.10. TESTING WITH AIKEN

Create `lib/zk_password/password_tests.ak` with two tests: one that asserts a valid proof passes, and one that asserts a wrong hash fails.

```
use aiken/crypto/bls12_381/scalar
use zk_password/groth16.{Proof, VerificationKey, groth16_verify}

fn test_vk() -> VerificationKey {
  VerificationKey {
```

```
    alpha: #"<vk_alpha1 from compressed_vk.json>",
    beta:  #"<vk_beta2  from compressed_vk.json>",
    gamma: #"<vk_gamma2 from compressed_vk.json>",
    delta: #"<vk_delta2 from compressed_vk.json>",
    ic: [
      #"<vk_ic[0] from compressed_vk.json>",
      #"<vk_ic[1] from compressed_vk.json>",
    ],
  }
}

fn test_proof() -> Proof {
  Proof {
    pi_a: #"<pi_a from compressed_proof.json>",
    pi_b: #"<pi_b from compressed_proof.json>",
    pi_c: #"<pi_c from compressed_proof.json>",
  }
}

// Public signal from public.json (the hash snarkjs used to generate the proof)
const credential_hash: Int =
  9415434746508747181228645764267412889835803815708024285868314342086739058356

test groth16_verify_valid_proof() {
  expect Some(hash_scalar) = scalar.new(credential_hash)
  groth16_verify(test_vk(), test_proof(), [hash_scalar])
}

test groth16_verify_wrong_hash() fail {
  let wrong_hash = credential_hash + 1
  expect Some(hash_scalar) = scalar.new(wrong_hash)
  groth16_verify(test_vk(), test_proof(), [hash_scalar])
}
```

Fill in the placeholder hex strings with the values from your `compressed_vk.json` and `compressed_proof.json`. The `credential_hash` is the integer in `public.json`.

```
aiken check
```

Expected output:

```
Testing ...
  PASS  zk_password/password_tests :: groth16_verify_valid_proof
  PASS  zk_password/password_tests :: groth16_verify_wrong_hash
Summary: 2 passed, 0 failed
```

> The `groth16_verify_wrong_hash` test is marked `fail`, meaning it is expected to fail. Aiken marks it as passing precisely *because* the verification rejects the wrong hash — confirming the validator cannot be fooled.

6.11. OFF-CHAIN CODE WITH MESHJS

With the validator compiled and tested, we need off-chain code to build the lock and unlock transactions. We use **MeshJS**, a TypeScript SDK for Cardano.

```
npm install @meshsdk/core
```

Read the compiled blueprint and the compressed keys:

```
import fs from "fs";
import {
  MeshWallet,
  Transaction,
  resolveScriptHash,
  serializePlutusScript,
  applyParamsToScript,
} from "@meshsdk/core";
import blueprint from "./plutus.json" assert { type: "json" };
import compressedVk from "./compressed_vk.json" assert { type: "json" };
import compressedProof from "./compressed_proof.json" assert { type: "json" };

// Helper: hex string -> Mesh ByteString value
const hex = (s) => ({ alternative: 0, fields: [s] });

// VerificationKey as Plutus Data (mirrors the Aiken record field order)
const vkData = {
  alternative: 0,
  fields: [
    compressedVk.vk_alpha1,
    compressedVk.vk_beta2,
    compressedVk.vk_gamma2,
    compressedVk.vk_delta2,
    compressedVk.vk_ic,   // List<ByteArray>
  ],
};
```

Locking a UTxO

To lock funds, build a transaction that sends ADA to the script address with the datum encoding `(credential_hash, vk)`:

```
import { publicSignals } from "./public.json" assert { type: "json" };

const credentialHash = publicSignals[0]; // decimal string from public.json

// Datum: Constr(0, [Int(credentialHash), VerificationKey])
const datum = {
  alternative: 0,
  fields: [
```

```
    parseInt(credentialHash), // credential_hash as Int
    vkData,                   // vk: VerificationKey
  ],
};

const script = {
  code: blueprint.validators[0].compiledCode,
  version: "V3",
};

const scriptAddress = serializePlutusScript(script, undefined, 0);

const wallet = new MeshWallet({
  networkId: 0,  // 0 = testnet, 1 = mainnet
  fetcher: provider,
  submitter: provider,
  key: { type: "mnemonic", words: yourMnemonic },
});

const tx = new Transaction({ initiator: wallet })
  .sendLovelace(
    { address: scriptAddress, datum: { value: datum, inline: true } },
    "5000000"  // 5 ADA
  );

const signed = await wallet.signTx(await tx.build());
const txHash = await wallet.submitTx(signed);
console.log("Locked:", txHash);
```

The datum is stored **inline** on-chain. Anyone can read the Poseidon hash and the verification key from the UTxO. Neither reveals the password.

Unlocking a UTxO

To unlock, the spender must have generated a valid proof off-chain (using `gen_input.js` + `snarkjs groth16 fullprove` + `compress_proof.js`). They then build a transaction with the redeemer encoding the compressed proof:

```
// Redeemer: Constr(0, [Proof])
const redeemer = {
  data: {
    alternative: 0,
    fields: [{
      alternative: 0,
      fields: [
        compressedProof.pi_a,
        compressedProof.pi_b,
        compressedProof.pi_c,
      ],
    }],
```

```
  },
  budget: { mem: 14000000, cpu: 10000000000 },
};

// Fetch the locked UTxO at the script address
const utxos = await provider.fetchAddressUTxOs(scriptAddress);
const lockedUtxo = utxos[0];

const tx = new Transaction({ initiator: wallet })
  .redeemValue({
    value: lockedUtxo,
    script,
    redeemer,
  })
  .sendLovelace(await wallet.getChangeAddress(), "4800000");

const signed = await wallet.signTx(await tx.build(), true);
const txHash = await wallet.submitTx(signed);
console.log("Unlocked:", txHash);
```

When this transaction is submitted, the Cardano node evaluates the `password_lock` validator. The validator decompresses the proof and VK, calls `groth16_verify`, and approves the spend only if the pairing equation holds — which it will if and only if the spender knows the correct password.

> The password is used only *locally* to run `fullprove`. It never appears in the transaction, the redeemer, or anywhere on-chain. An observer who inspects the blockchain sees the compressed proof and the hash — neither reveals the password.

6.12. END-TO-END SUMMARY

Here is the complete workflow from password to unlocked UTxO:

1. **Setup** (once per circuit): compile `circuit.circom`, run the trusted setup ceremony, export `verification_key.json`, compress it to `compressed_vk.json` with `compress_vk.js`.
2. **Lock**: choose a password, run `node circuit/gen_input.js <password>` to produce `circuit/input.json`, then lock a UTxO with the datum `(credential_hash, vk)`.
3. **Prove**: run `snarkjs groth16 fullprove` with the `input.json`, the WASM, and `zkey_final.zkey` to produce `proof.json`.
4. **Compress**: run `compress_proof.js` to get `compressed_proof.json`.
5. **Unlock**: build a transaction spending the UTxO with the redeemer encoding the compressed proof.

6.13. HOW THIS DIFFERS FROM JANUS WALLET

Janus Wallet is a production ZK wallet with staking, delegation, and a server-side proving API. This chapter's contract is intentionally stripped down to isolate the core

idea.

Feature	Janus Wallet	zk-password
Public inputs	userId, hash, challenge, flags	hash only
Replay protection	On-chain challenge nonce	None (educational)
Server	Node.js + PostgreSQL	Off-chain only
Hash function	Poseidon (Circom)	Poseidon (Circom)
Curve	BLS12-381	BLS12-381
On-chain verifier	Aiken + builtins	Aiken + builtins
Staking / governance	Yes	No

The `zk-password` contract does not include replay protection. In production, a malicious observer could copy a valid proof from the blockchain and resubmit it in their own transaction before the legitimate spender. To prevent this, add a nonce or challenge to the circuit's public inputs and store an on-chain counter in the datum, as Janus Wallet does with its `challenge` and `challengeFlag` signals.

This is the complete minimal ZK contract. The next chapters will add replay protection, multi-input circuits, and a full off-chain proving workflow.